From Synthetic Benchmarks to a Live Network: Deploying and Auditing a Lightweight Flow-Based IoT Intrusion Detector on a Raspberry Pi 2

Mahi Patel
Dept. of Computer Engineering
COEP Technological University
Pune, India
[TBD: e-mail]

Malhar Falke
Dept. of Computer Engineering
COEP Technological University
Pune, India
[TBD: e-mail]

Yugandhar Pise
Dept. of Computer Engineering
COEP Technological University
Pune, India
[TBD: e-mail]

Vaibhav Tayade
Dept. of Computer Engineering
COEP Technological University
Pune, India
[TBD: e-mail]

S. K. Gaikwad
Dept. of Computer Engineering
COEP Technological University
Pune, India
[TBD: e-mail]

***Abstract*—Machine-learning intrusion detectors for Internet of Things (IoT) networks are usually reported with near-perfect accuracy on benchmark data, yet few are measured after deployment on the constrained hardware they target. We built a complete flow-based detector and followed it from a synthetic benchmark onto a live network. A single 22-feature extractor serves both training and detection, and a 92 KB gradient-boosted tree model classifies ten traffic classes. Because no 32-bit ARM build of the usual inference runtime exists, we compiled the model to dependency-free C and ran it on a Raspberry Pi 2, where it reproduced the reference labels on 5,000 inputs. On synthetic held-out traffic the model scored 99.65% accuracy, but removing features and changing the split did not move this score, which shows that the generated classes are trivially separable rather than that the detector is accurate. On the Raspberry Pi 2, all seven controlled attacks raised an incident within 21 s, and the sensor ran for 14.4 h with no restart and no interface packet loss. Ordinary home-network traffic, however, produced 1.9–2.6 false-alarm incidents per minute, and confidence thresholds did not remove them. A post-hoc incident-size rule removed every benign incident but also two of the seven attacks. A corrected cross-dataset study over eleven datasets, fixing a leakage error in our earlier one, finds a mean area under the receiver operating characteristic curve of 0.979 within a dataset but 0.497 across datasets. The results argue that edge intrusion detectors should be evaluated on representative benign traffic before any accuracy figure is reported.**

***Index Terms*—Internet of Things, intrusion detection, edge computing, Raspberry Pi, gradient boosting, false alarms, evaluation methodology**

I. Introduction

IoT devices are attractive targets because they are numerous, rarely patched, and often reachable with default credentials; the Mirai botnet showed how quickly such devices can be recruited at scale [1]. Network intrusion detection is a natural defence, and machine learning (ML) has become its dominant research approach [2]–[4]. Published detectors routinely report accuracy above 99% on public datasets. Two gaps remain between these figures and a working sensor.

First, the hardware gap. A detector intended for an IoT gateway must run on a few hundred megabytes of memory and a low-power CPU, often without the runtimes used during development. Many studies measure inference speed on a workstation and leave deployment as future work.

Second, the evaluation gap. Security ML is prone to pitfalls such as sampling bias, data leakage between training and test data, and lab-only evaluation [5]–[7]. Public intrusion datasets also contain labelling and construction artefacts [8], [9]. A model can then score highly by learning how a dataset was made rather than what an attack looks like. Benign traffic is the critical case: a sensor sees benign traffic almost all the time, so even a small false-positive rate per flow produces a stream of alerts.

This paper follows one lightweight detector across both gaps. Our contributions are:

1) An end-to-end, open flow-based IoT detector in which one packet-to-flow feature extractor serves both training and live detection, and a compact tree model is exported to dependency-free C. We show that the C model reproduces the reference labels exactly on a 32-bit Raspberry Pi 2.
2) A deployment measurement on that board: inference latency, packet parsing throughput, capture loss under floods, memory, temperature, reboot recovery, and a 14.4-hour soak.
3) An evaluation audit. We show that a near-perfect synthetic score is uninformative, and measure what happens on a live home network: controlled attacks are detected, but benign traffic triggers a sustained false-alarm rate

that confidence thresholds do not remove. We quantify the trade-off of a simple incident-size filter.

4) A corrected cross-dataset study over eleven public datasets, including a report of the resubstitution error that invalidated our own earlier result. With a held-out diagonal, detectors still transfer at chance on average (mean area under the receiver operating characteristic curve, ROC-AUC, 0.497).

Section II reviews related work. Section III describes the system and Section IV the evaluation method. Section V presents the results, Section VI discusses their implications and limitations, and Section VII concludes.

II. Related Work

IoT intrusion detection. Surveys group IoT detectors by placement, detection method and validation strategy, and note that most proposals are validated by simulation or on offline datasets rather than in deployment [2]–[4]. Lightweight online detectors exist: Kitsune trains an ensemble of small autoencoders on per-packet statistics and runs on a Raspberry Pi [10], and N-BaIoT detects botnet traffic per device with deep autoencoders [11]. Our work differs in that it uses a supervised multiclass flow model, exports it to plain C, and measures benign false alarms on a live network.

Evaluation pitfalls. Sommer and Paxson argued that intrusion detection differs from other ML tasks because errors are costly and benign traffic is highly diverse [5]. Arp *et al.* catalogued pitfalls including data snooping, spurious correlations and lab-only evaluation [6], and Apruzzese *et al.* call for assessments that reflect operational constraints [7]. Leakage, where information about the test data reaches training, is a general cause of over-optimistic results [12]. We encountered both spurious separability and leakage in our own pipeline and report them.

Cross-dataset generalisation. Flow datasets use different feature extractors, such as CICFlowMeter [13] or Zeek, which complicates comparison. Sarhan *et al.* proposed a standard NetFlow-based feature set so that detectors can be compared across datasets [14]. Our cross-dataset study instead aligns eleven datasets' native features into a common twelve-feature space by meaning and unit (Section IV-D).

III. System Design

A. Flow features

The pipeline is shown in Fig. 1. Packets are parsed from a capture file or a live interface (Ethernet, VLAN, IPv4, IPv6, TCP, UDP and ICMP) and grouped into bidirectional flows keyed by protocol and endpoints. Each flow is summarised by 22 features: protocol, duration, packet and byte counts in each direction, packet and byte rates, packet-length and inter-arrival statistics, TCP SYN/FIN/RST/ACK ratios, direction asymmetry, the destination service port, and three *host-context* features that count the distinct destination ports and hosts contacted by the source in a rolling window. Host context separates attacks that look identical at the level of a single flow: a port scan touches many ports on one host, a Mirai-style propagation touches one port on many hosts, and a flood sends heavy traffic to one host and port. The feature layout is versioned, and the detector refuses a model built for a different layout.

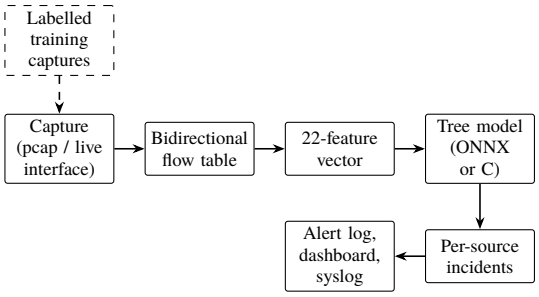


Fig. 1. Detection pipeline. Training captures (dashed) pass through the same capture-to-feature path as live traffic, so there is no separate training-time feature code.

B. Model and export

The classifier is an XGBoost multiclass model [15] over ten classes: benign, three reconnaissance or brute-force classes (port scan, Xmas/NULL/FIN scan, SSH brute force), five denial-of-service classes (SYN, UDP, ICMP and MQTT floods, and slowloris) and a Mirai-style botnet class. It has 1,200 trees with 2,780 nodes in total. Trees are invariant to feature scaling, so no scaler is shipped. The model is exported to ONNX (91.8 KB) for ONNX Runtime [16] and, by our own exporter, to a single C99 header containing about 49 KB of constant node data. The C inference function uses no heap and about 130 bytes of stack. The exporter verifies that the C model makes the same decisions as the XGBoost model.

C. Incidents

Per-flow verdicts are aggregated into incidents keyed by source address and attack type, so a 700-port scan produces one incident rather than 700 alerts. An incident is reported when a flow of that type exceeds a confidence threshold (0.5 by default). Incidents are written to a log, a local web dashboard, and optionally a syslog collector. An experimental response module can rate-limit or block a source, but it is disabled in all results below except one experiment in Section VI.

IV. Evaluation Method

A. Synthetic training corpus

Labelled IoT packet captures with a ground truth that can be joined to our own flow extractor were not available to us. We therefore generated traffic with Scapy [17]: each scenario mixes benign background traffic with one attack, and a "hard benign" family produces bursty transfers and multi-host telemetry that resemble attacks. The corpus has 350 scenarios. We split it *by scenario* (280 for training and 70 for testing) so that no attacker, address set or timing pattern appears on both sides; this gives 44,761 training flows and 12,191 test flows. Two further synthetic sets were generated

from unseen random seeds for validation and benchmarking. We report all three separately.

B. Deployment board and live network

The deployment board is a Raspberry Pi 2 Model B (BCM2836, four Cortex-A7 cores, 1 GB RAM) running 32-bit Raspberry Pi OS (Debian 13, Python 3.13). ONNX Runtime publishes no wheels for 32-bit ARM, so the board runs the C model through a thin `ctypes` wrapper. To check that this changes nothing, we also built ONNX Runtime 1.23.2 for armv7 from source. The sensor was installed as a system service with its default settings. The live trial used an ordinary home network (`/24` IPv4 subnet with IPv6) and devices owned by the authors only.

C. Live experiments

- *Controlled attacks.* Seven attacks were run one at a time with standard tools: a TCP connect scan and an Xmas scan (`nmap`), SYN, UDP and ICMP floods at about 2,000 packets/s for 20 s (`hping3`), 40 failed SSH logins, and slowloris with 40 held connections. Network attacks came from a laptop on the same network. The floods and the Xmas scan were run from the Pi against itself on the loopback interface so that no flood left the board.
- *Benign baselines.* Two ten-minute windows with no attacks: the first observed live, the second captured with `tcpdump` (headers only) and replayed through the same detector so that distinct flows could be counted.
- *Soak.* The service was sampled every five minutes for 14.4 h (service state, restarts, memory, temperature, throttling, interface drops, packet and incident counters).
- *Reboot.* Time from a reboot command to a fresh sensor heartbeat.

D. Cross-dataset protocol

For the research track we aligned eleven public flow datasets [18]–[28] into twelve shared features by meaning and unit: durations are converted to seconds, missing rates are derived from counts and duration where possible, and a feature a dataset cannot provide is left missing rather than filled with zero, because a constant column would reveal the dataset's identity. For each dataset a binary model is trained on a fixed stratified 80% split. The diagonal of the resulting 11×11 matrix is measured on the untouched 20%; every off-diagonal cell tests on a different dataset. We report ROC-AUC and the Matthews correlation coefficient instead of F1, because on a balanced test set a classifier that labels everything as an attack already scores F1 $= 0.667$.

V. Results

A. Synthetic accuracy is uninformative

Table I lists the three synthetic evaluations. The only errors are in the benchmark set, where Mirai-style traffic has 94.2% recall; the attack detection rate is 100% and no benign flow is flagged. We do not read these figures as detector accuracy. Splitting by scenario instead of by row, mixing benign background into every attack scenario, and removing the destination port changed accuracy by at most 10^{-4}. When every check leaves the score at its ceiling, the score cannot distinguish a good detector from a poor one: the generated classes are separable along several redundant features at once. The ablations are therefore uninformative on this corpus, and the synthetic results show only that the pipeline works end to end.

TABLE I
Three Synthetic Evaluations of the Same Model

Evaluation set	Flows	Accuracy	Macro-F1
Scenario-held-out test split	12,191	100.00%	1.000
Validation, unseen seeds	47,485	100.00%	1.000
Benchmark, unseen seeds	35,057	99.65%	0.996
Test split without `dst_port`	12,191	100.00%	1.000

TABLE II
Inference and Throughput on the Raspberry Pi 2 and a Reference Host

Measure	Pi 2 (C model)	Host†
Single flow (µs)‡	535	9.7
p99 (µs)	799	23.9
Batched inference (flows/s, batch 1024)	17,506	408,321
Bare C model (µs/flow)	50.8	1.6
Packet parsing (packets/s)	6,758	129,003
End to end, capture to verdict (flows/s)	1,316	35,575
Sensor memory (RSS, MB)	39–70	57.8

†Intel Xeon at 2.30 GHz (cloud VM), ONNX Runtime; shown for scale only. ‡Pi: full Python detector call; host: ONNX Runtime call.

B. Inference on 32-bit ARM

On 5,000 inputs, the C model on the Pi and ONNX Runtime on a 64-bit host produced identical labels, with a largest probability difference of 1.07×10^{-6}. The armv7 build of ONNX Runtime also agreed on every label. It was faster for a single flow (331 vs. 511 µs on an idle board) but less than half as fast in batches of 1,024 (7,881 vs. 17,492 flows/s), so the C model remains the default. It also needs no custom-built runtime.

Table II shows that the model itself is not the bottleneck. Most of the 535 µs single-flow time is the Python call and post-processing; the bare C tree walk takes 50.8 µs. The limit is packet parsing in Python, at about 6,800 packets/s. This is ample for a quiet IoT segment but not for a flood: during the 2,000 packets/s floods, the sensor processed only about 12–14% of the offered packets.

C. Controlled attacks

Table III lists the main incident raised for each attacker. All seven attacks raised an incident within 21 s, and five of the seven carried the correct type at a confidence of at least 0.9. The UDP flood was detected as an attack but labelled as an Xmas scan. The ICMP flood was detected late and weakly. The SSH brute force produced only two flows of roughly 40 connections, so the attack was detected but most of its activity was not seen.

TABLE III
CONTROLLED ATTACKS ON THE RASPBERRY PI 2

Attack (tool)	Reported type	Flows	Conf.	Alert
TCP port scan (nmap)	port scan	1,539	1.00	1 s
SSH brute force	SSH brute force	2	1.00	21 s
Slowloris	slowloris	29	0.92	1 s
Xmas scan (nmap)	Xmas scan	710	0.97	3 s
SYN flood (hping3)	SYN flood	2,618	0.99	2 s
UDP flood (hping3)	*Xmas scan*	3,288	0.86	<1 s
ICMP flood (hping3)	ICMP flood	2	0.52	13 s

TABLE IV
FALSE ALARMS ON BENIGN HOME-NETWORK TRAFFIC

	Baseline 1	Baseline 2
Duration	10 min	10 min
Packets	2,475	3,696
Distinct flows	—	69
Flows labelled as an attack	—	48
False-alarm incidents	19	26
per minute	1.9	2.6
from IPv6 sources	11	16
remaining at confidence ≥ 0.9	13	16
remaining at ≥ 5 flows	0	0

D. Benign traffic and false alarms

Table IV shows the central result. With no attacks running, the detector raised 19 and 26 false-alarm incidents in two ten-minute windows. In the second window it labelled 48 of the 69 distinct flows as an attack at least once. Most were classed as UDP or ICMP floods, and most came from IPv6 sources, many of them link-local; the router itself was one of the sources. This is ordinary network chatter, such as IPv6 neighbour discovery, multicast and service announcements, that the synthetic corpus never contained.

Confidence did not help: 13 of 19 and 16 of 26 incidents had a confidence of at least 0.9. As tree-model and neural-network scores are not calibrated probabilities [29], a high score on data unlike the training set carries little meaning. Incident size did separate the two windows: no benign incident contained more than three flows. However, the same rule applied to Table III would also suppress the SSH brute force and the ICMP flood, which had two flows each. A minimum of five flows per incident would therefore have removed every observed false alarm at the cost of two of seven attacks. Because this threshold was chosen after looking at the same data, it is a hypothesis to be tested on new sessions, not a validated filter.

E. Stability

Fig. 2 shows the soak. The service stayed active in all 173 samples with no restart, the network interface reported no dropped packets among about 98,800 received, and the CPU stayed between 50.3 and 55.1 °C with no thermal throttling. Memory rose from 53.5 to 62.4 MB over the first seven hours and by only 0.6 MB over the last seven; a longer run is needed to rule out a slow leak. The incident counter rose by 933 over the window, about 65 per hour, which confirms that the false-alarm rate of Table IV persists for hours. After a reboot, the board accepted SSH after 80 s and the sensor reported a fresh heartbeat after 99 s without manual action.

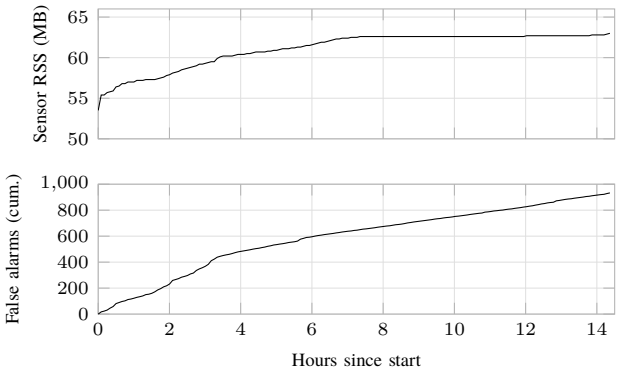


Fig. 2. The 14.4-hour soak on the Raspberry Pi 2 (five-minute samples). Top: sensor memory. Bottom: cumulative incidents with no attacks running.

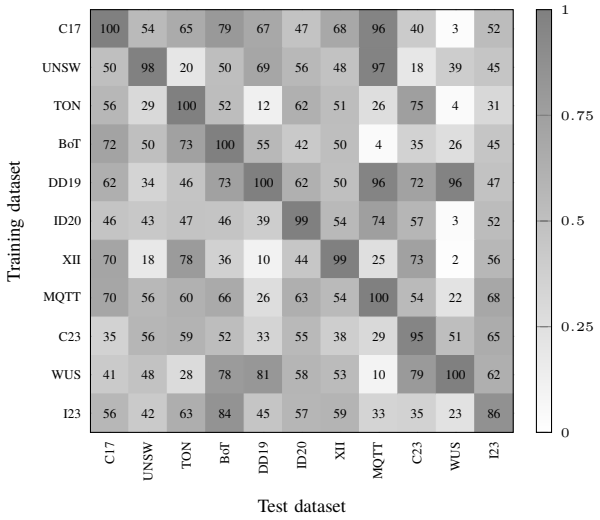


Fig. 3. Cross-dataset ROC-AUC ($\times 100$) after the correction. The diagonal is measured on a held-out 20% split; every other cell tests on a different dataset. 50 is chance; values below 50 are inverted rankings. C17 CICIDS2017, UNSW UNSW-NB15, TON TON_IoT, BoT Bot-IoT, DD19 CICDDoS2019, ID20 IoTID20, XII X-IIoTID, MQTT MQTT-IoT-IDS2020, C23 CIC-IoT-2023, WUS WUSTL-IIoT-2021, I23 IoT-23. BoT has only 11 benign flows in its held-out split.

F. Cross-dataset transfer

Our earlier cross-dataset study reported a near-perfect in-domain score and a large gap to cross-domain performance. During a code review we found that the in-domain cells had been evaluated on the same rows used for training, so the in-domain score and the gap were both invalid. We withdrew them and re-implemented the protocol of Section IV-D. Each dataset contributed 50,000 flows; each model was trained on 40,000 and its diagonal cell measured on the remaining 10,000.

Fig. 3 shows the corrected matrix. The in-domain cells average ROC-AUC 0.979 (median 0.998) and MCC 0.898. The 110 cross-domain cells average ROC-AUC 0.497 and MCC -0.014, which is chance. The gap of 0.482 is close to the withdrawn figure, but it now rests on a held-out diagonal. Of the 110 cells, 65 have ROC-AUC at most 0.55, and 50 fall below 0.5, down to 0.016. These are not random failures: a feature direction that marks attacks in one dataset marks benign traffic in another, and the ten other models average 0.267 on WUSTL-IIoT. Only four cells reach 0.9, and their MCC is at most 0.46, so even these need a new decision threshold. Bot-IoT is 99.9% attack traffic, which leaves only 11 benign flows in its held-out split; without Bot-IoT on either axis the means are 0.977 in domain and 0.489 across domains, so it does not change the result.

Pooling five datasets for training (Bot-IoT included, 250,000 flows) did not close the gap. On the six unseen datasets ROC-AUC ranged from 0.347 to 0.979: MQTT-IoT-IDS2020 (0.979) and CICDDoS2019 (0.930) transferred, while on WUSTL-IIoT and IoT-23 the pooled model flagged 99.5% and 96.5% of benign flows. Re-fitting the threshold on ten labelled target flows raised MCC from 0.300 to 0.764 on CICDDoS2019 and from 0.788 to 0.909 on MQTT-IoT-IDS2020 (means of 20 repeats), but no threshold helped on the other four, where the ranking itself carries no signal. In leave-one-dataset-out training on all eleven datasets, no feature transform that can be exported to the microcontroller improved on standard scaling (mean ROC-AUC 0.612). A quantile transform reached 0.646, but its lookup tables are too large for the C export.

VI. Discussion

What the deployment showed. The detector runs on a 32-bit board with ten-year-old CPU cores, survives reboots, and remained stable for 14.4 h. On the attack side, the host-context features worked as designed for scans and SYN floods. The failures were on the benign side. A sensor on a quiet home network produced about one false alarm per minute, which an operator would not tolerate, and the model's confidence offered no way to separate these alarms from real attacks. None of this was visible in the synthetic evaluation, where the benign false-positive rate was zero.

Why prevention was not enabled. In one isolated experiment we enabled the response module. It blocked the scanning laptop within about 5 s and lifted the block after the scan ended, but it also throttled twelve innocent sources, including the router. Automatic blocking turns every false alarm into an outage, so we treat the false-alarm rate as the precondition for any response mechanism and report detection only.

Lessons for evaluation. Our experience supports the pitfalls described in [5]–[7]. A ceiling score is not evidence of accuracy, and ablations that cannot move the score are uninformative. Leakage can enter through evaluation code as easily as through data, as our resubstitution error showed. In addition, a sensor's dominant input is benign traffic, so benign false alarms per unit time are the measure that decides whether a detector is usable.

Limitations. The model was trained on generated traffic only. The live trial used one home network, two short benign windows and seven attacks, which is evidence of behaviour rather than a statistical estimate of accuracy. The floods and the Xmas scan ran over the loopback interface. The soak lasted 14.4 h rather than 24 h. The cross-dataset matrix comes from one seed and one 50,000-flow sample per dataset, so it carries no confidence intervals. The incident-size rule was chosen post hoc. Packet capture in Python limits throughput under floods. No device other than the sensor was protected.

Future work. The first step is to train on real benign captures from IoT networks and to select alert filters on separate calibration sessions. Parsing IPv6 extension headers, excluding link-local and multicast sources from incident reporting, and moving capture to a compiled front end are smaller, direct improvements.

VII. Conclusion

We built a lightweight flow-based IoT intrusion detector, compiled it to dependency-free C, and deployed it on a Raspberry Pi 2. It reproduced the reference model's labels exactly, detected all seven controlled attacks, and ran stably for 14.4 h. Its near-perfect synthetic accuracy, however, did not predict its behaviour on a real network, where ordinary benign traffic produced a sustained false-alarm rate that confidence thresholds could not remove. Across eleven public datasets, the corrected study found that a detector trained on one dataset ranks another at chance on average. Edge intrusion detectors should therefore be measured on representative benign traffic, in false alarms per unit time, before accuracy figures are reported or automatic prevention is enabled. The code, model, raw measurements and analysis scripts are available in the project repository [TBD: repository URL, if the team makes it public].

Acknowledgment

[TBD: Acknowledge the department and any lab resources used.]

References

[1] M. Antonakakis *et al.*, "Understanding the Mirai botnet," in *Proc. 26th USENIX Security Symp.*, Vancouver, BC, Canada, 2017, pp. 1093–1110.

[2] B. B. Zarpelão, R. S. Miani, C. T. Kawakani, and S. C. de Alvarenga, "A survey of intrusion detection in Internet of Things," *Journal of Network and Computer Applications*, vol. 84, pp. 25–37, 2017.

[3] S. Hajiheidari, K. Wakil, M. Badri, and N. J. Navimipour, "Intrusion detection systems in the Internet of Things: A comprehensive investigation," *Computer Networks*, vol. 160, pp. 165–191, 2019.

[4] A. Khraisat and A. Alazab, "A critical review of intrusion detection systems in the Internet of Things: techniques, deployment strategy, validation strategy, attacks, public datasets and challenges," *Cybersecurity*, vol. 4, no. 1, p. 18, 2021.

[5] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in *Proc. IEEE Symp. Security and Privacy*, Oakland, CA, USA, 2010, pp. 305–316.

[6] D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressnegger, L. Cavallaro, and K. Rieck, "Dos and don'ts of machine learning in computer security," in *Proc. 31st USENIX Security Symp.*, Boston, MA, USA, 2022, pp. 3971–3988.

[7] G. Apruzzese, P. Laskov, and J. Schneider, "SoK: Pragmatic assessment of machine learning for network intrusion detection," in *Proc. IEEE 8th European Symp. Security and Privacy (EuroS&P)*, Delft, Netherlands, 2023, pp. 592–614.

[8] G. Engelen, V. Rimmer, and W. Joosen, "Troubleshooting an intrusion detection dataset: the CICIDS2017 case study," in *Proc. IEEE Security and Privacy Workshops (SPW)*, San Francisco, CA, USA, 2021, pp. 7–12.

[9] A. Kenyon, L. Deka, and D. Elizondo, "Are public intrusion datasets fit for purpose? characterising the state of the art in intrusion event datasets," *Computers & Security*, vol. 99, p. 102022, 2020.

[10] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, "Kitsune: An ensemble of autoencoders for online network intrusion detection," in *Proc. Network and Distributed System Security Symp. (NDSS)*, San Diego, CA, USA, 2018.

[11] Y. Meidan, M. Bohadana, Y. Mathov, Y. Mirsky, A. Shabtai, D. Breitenbacher, and Y. Elovici, "N-BaIoT—network-based detection of IoT botnet attacks using deep autoencoders," *IEEE Pervasive Computing*, vol. 17, no. 3, pp. 12–22, 2018.

[12] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, "Leakage in data mining: Formulation, detection, and avoidance," *ACM Trans. Knowledge Discovery from Data*, vol. 6, no. 4, pp. 15:1–15:21, 2012.

[13] A. H. Lashkari, G. Draper-Gil, M. S. I. Mamun, and A. A. Ghorbani, "Characterization of Tor traffic using time based features," in *Proc. 3rd Int. Conf. Information Systems Security and Privacy (ICISSP)*, Porto, Portugal, 2017, pp. 253–262.

[14] M. Sarhan, S. Layeghy, and M. Portmann, "Towards a standard feature set for network intrusion detection system datasets," *Mobile Networks and Applications*, vol. 27, no. 1, pp. 357–370, 2022.

[15] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, San Francisco, CA, USA, 2016, pp. 785–794.

[16] ONNX Runtime developers, "ONNX Runtime," [Online]. Available: https://onnxruntime.ai, 2021, accessed: Nov. 2026.

[17] P. Biondi and the Scapy community, "Scapy: Packet crafting for Python," [Online]. Available: https://scapy.net, accessed: Nov. 2026.

[18] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. 4th Int. Conf. Information Systems Security and Privacy (ICISSP)*, Funchal, Portugal, 2018, pp. 108–116.

[19] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems," in *Proc. Military Communications and Information Systems Conf. (MilCIS)*, Canberra, Australia, 2015, pp. 1–6.

[20] A. Alsaedi, N. Moustafa, Z. Tari, A. Mahmood, and A. Anwar, "TON_IoT telemetry dataset: A new generation dataset of IoT and IIoT for data-driven intrusion detection systems," *IEEE Access*, vol. 8, pp. 165 130–165 150, 2020.

[21] N. Koroniotis, N. Moustafa, E. Sitnikova, and B. Turnbull, "Towards the development of realistic botnet dataset in the Internet of Things for network forensic analytics: Bot-IoT dataset," *Future Generation Computer Systems*, vol. 100, pp. 779–796, 2019.

[22] E. C. P. Neto, S. Dadkhah, R. Ferreira, A. Zohourian, R. Lu, and A. A. Ghorbani, "CICIoT2023: A real-time dataset and benchmark for large-scale attacks in IoT environment," *Sensors*, vol. 23, no. 13, p. 5941, 2023.

[23] I. Sharafaldin, A. H. Lashkari, S. Hakak, and A. A. Ghorbani, "Developing realistic distributed denial of service (DDoS) attack dataset and taxonomy," in *Proc. Int. Carnahan Conf. Security Technology (ICCST)*, Chennai, India, 2019, pp. 1–8.

[24] I. Ullah and Q. H. Mahmoud, "A scheme for generating a dataset for anomalous activity detection in IoT networks," in *Advances in Artificial Intelligence (Canadian AI 2020)*, ser. Lecture Notes in Computer Science, vol. 12109. Springer, 2020, pp. 508–520.

[25] M. Al-Hawawreh, E. Sitnikova, and N. Aboutorab, "X-IIoTID: A connectivity-agnostic and device-agnostic intrusion data set for industrial Internet of Things," *IEEE Internet of Things Journal*, vol. 9, no. 5, pp. 3962–3977, 2022.

[26] H. Hindy, E. Bayne, M. Bures, R. Atkinson, C. Tachtatzis, and X. Bellekens, "Machine learning based IoT intrusion detection system: An MQTT case study (MQTT-IoT-IDS2020 dataset)," in *Selected Papers from the 12th Int. Networking Conf. (INC 2020)*, ser. Lecture Notes in Networks and Systems, vol. 180. Springer, 2021, pp. 73–84.

[27] M. Zolanvari, M. A. Teixeira, L. Gupta, K. M. Khan, and R. Jain, "WUSTL-IIOT-2021 dataset for IIoT cybersecurity research," Washington Univ. in St. Louis. [Online]. Available: https://www.cse.wustl.edu/~jain/iiot2/index.html, 2021.

[28] S. Garcia, A. Parmisano, and M. J. Erquiaga, "IoT-23: A labeled dataset with malicious and benign IoT network traffic," Zenodo, 2020, doi: 10.5281/zenodo.4743746.

[29] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proc. 34th Int. Conf. Machine Learning (ICML)*, Sydney, Australia, 2017, pp. 1321–1330.